

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Дисциплина: Архитектура ЭВМ

Тема: Создание и процесс обработки программ на языке ассемблера NASM

Студент: Али Мухтар Хадир

Группа: НПИ-01

Студенческий билет: 1132254991

Преподаватель: Демидова А. В.

1. Цель работы

- Освоение процедуры компиляции и сборки программ, написанных на ассемблере NASM.
- Изучение основных принципов работы процессора и использования системных вызовов в операционной системе Linux.

2. Выполнение работы

2.1. Подготовка рабочего пространства

Я создал отдельный каталог для выполнения четвертой лабораторной работы и подготовил файл для исходного кода.

- Команда: `mkdir -p ~/work/arch-pc/lab04`
- Команда: `cd ~/work/arch-pc/lab04`
- Команда: `touch hello.asm`
- Результат: Рабочее окружение настроено, файл `hello.asm` создан.

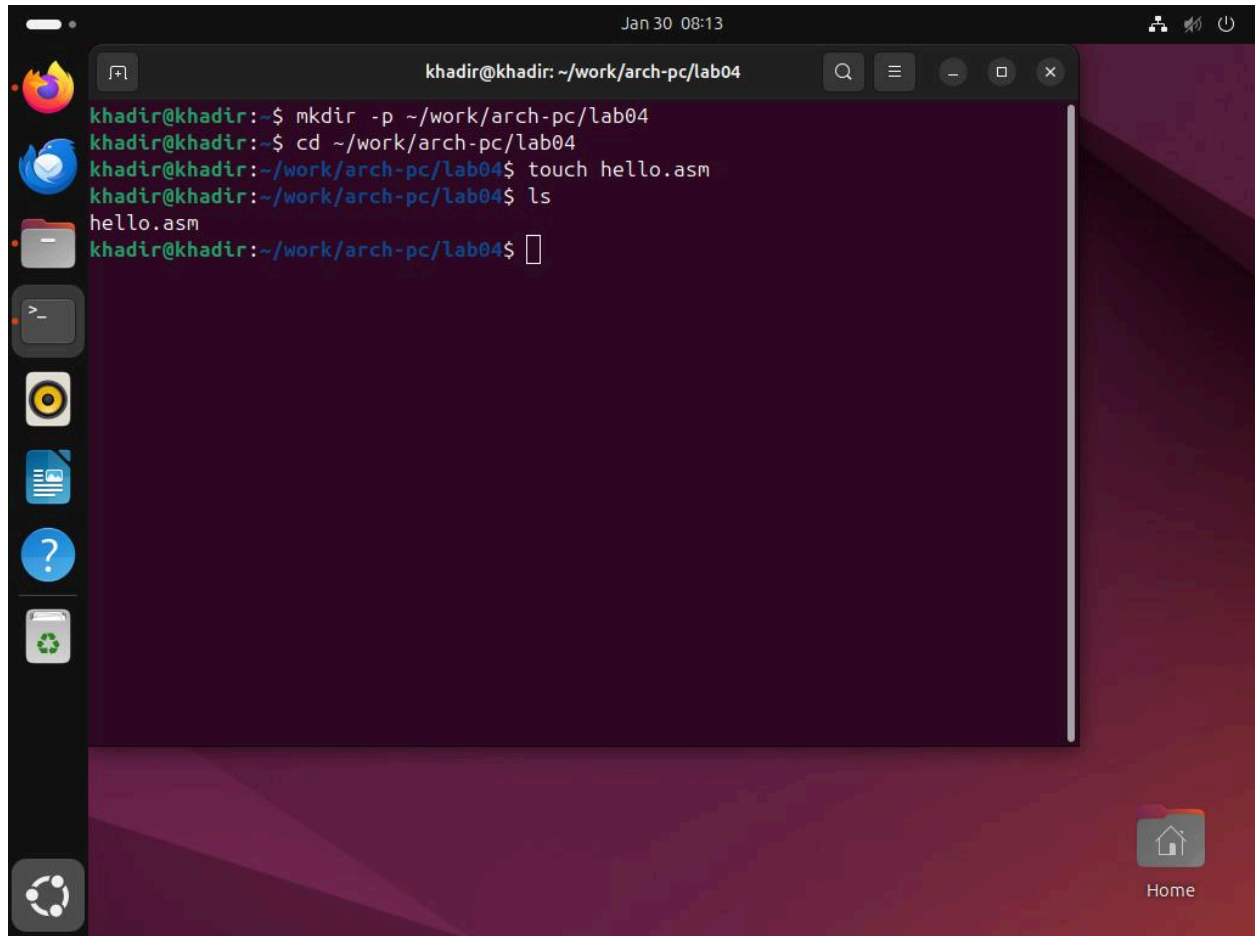


Рис. 4.1. Подготовка рабочего каталога.

2.2. Написание исходного текста программы

Я открыл файл в текстовом редакторе `gedit` и ввел текст программы "Hello World". Программа использует регистры процессора для вывода строки на экран.

- **Команда:** `gedit hello.asm`
- **Действие:** Ввод кода с использованием системного вызова `sys_write` (`eax=4`) и завершением программы (`eax=1`).
- **Результат:** Исходный код сохранен и готов к трансляции.

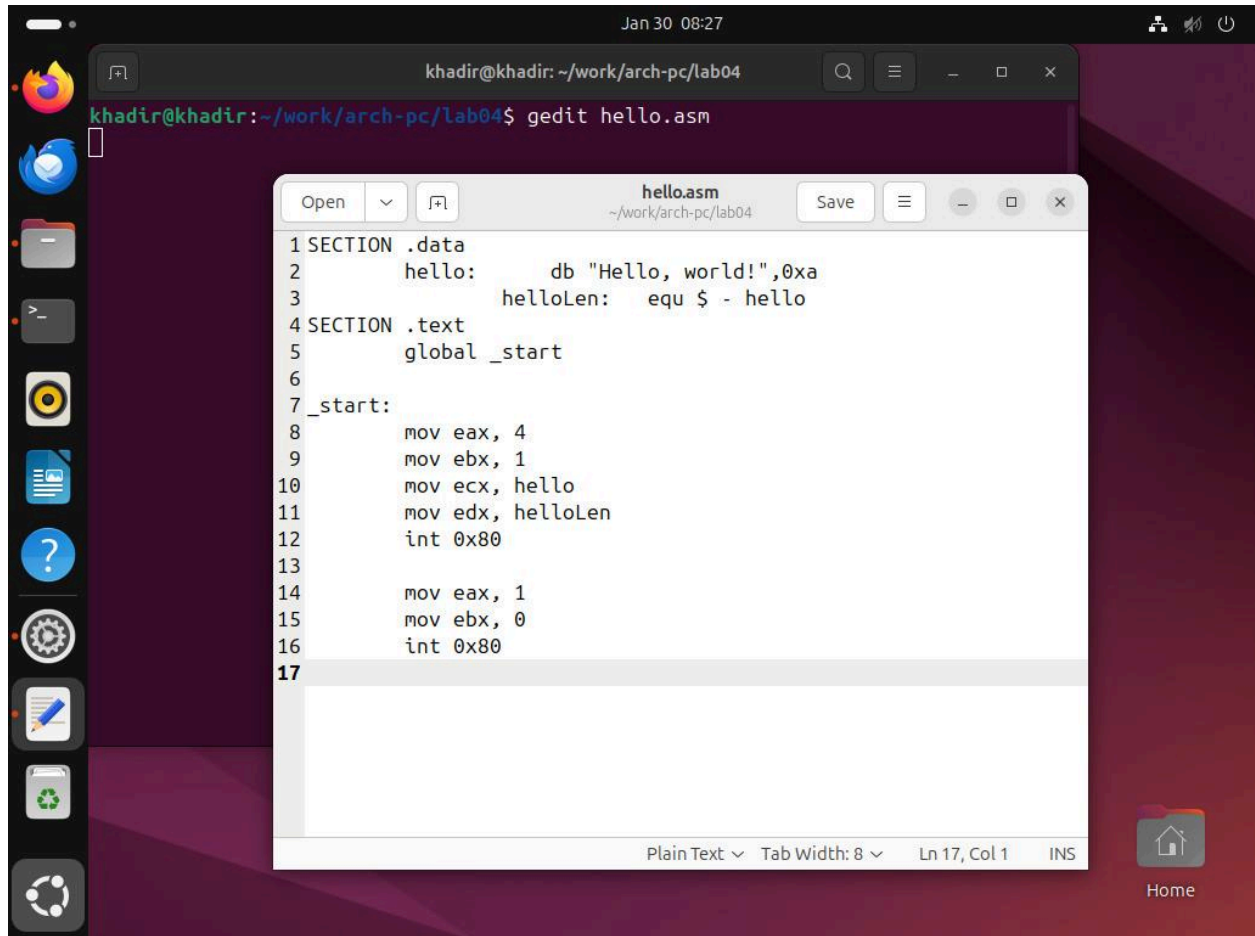


Рис. 4.2. Редактирование исходного кода.

2.3. Трансляция и компоновка

Для получения исполняемого файла я использовал транслятор *nasm* и компоновщик *ld*. Так как работа проводится на архитектуре **ARM64**, я использовал кросс-компоновщик для сборки 32-битного приложения *x86*.

- **Команда:** *nasm -f elf hello.asm -o hello.o*
- **Команда:** *i686-linux-gnu-ld -m elf_i386 hello.o -o hello*
- **Команда:** *qemu-i386 ./hello*
- **Результат:** Программа успешно вывела строку "Hello, world!" в терминал.

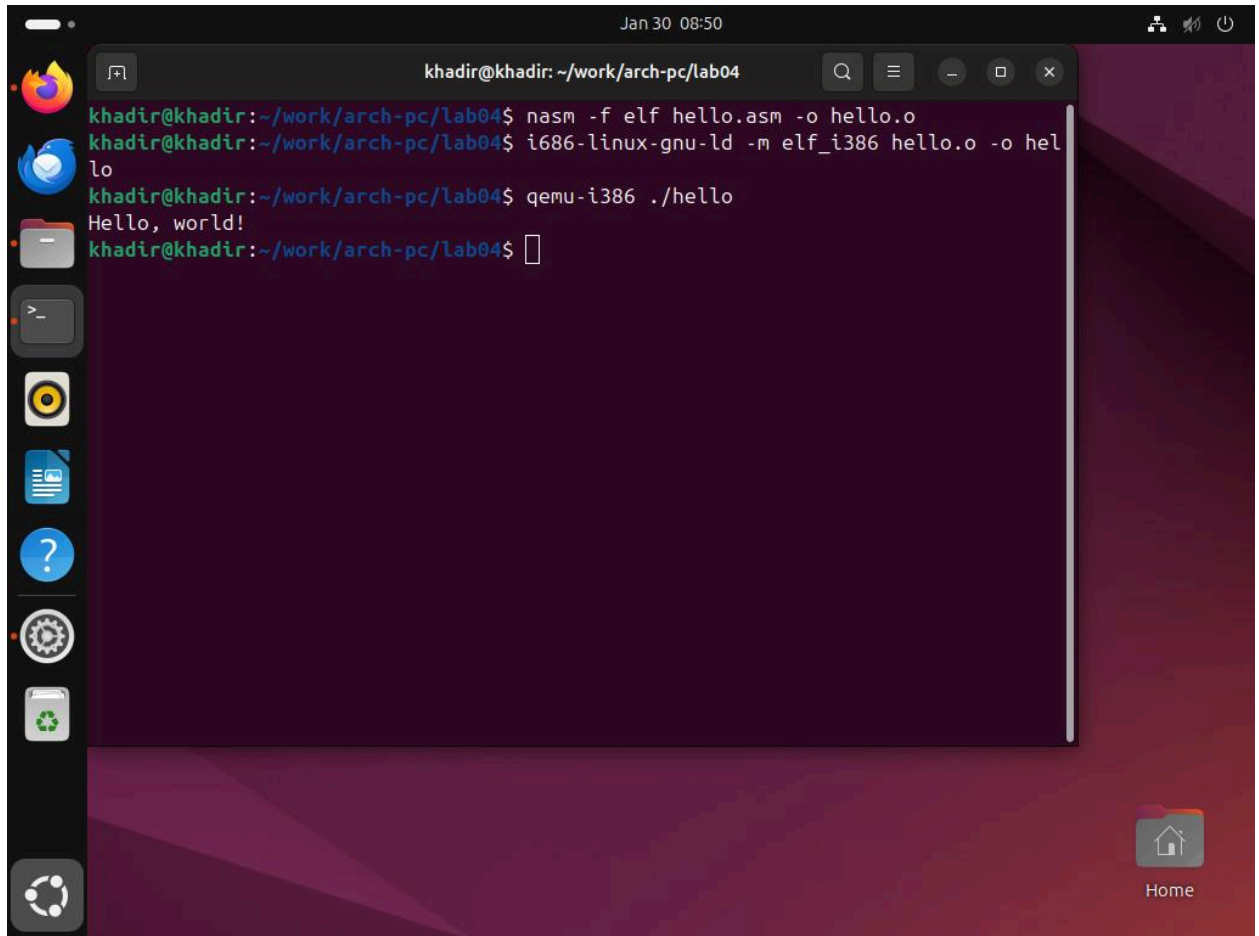


Рис. 4.3. Трансляция, компоновка и запуск программы через эмулятор `qemu-i386`.

Согласно заданию 4.5, я создал копию программы и изменил её так, чтобы она выводила мою фамилию и имя. Для сборки и запуска на архитектуре ARM64 использовались кросс-компоновщик и эмулятор.

- **Команда:** `cp hello.asm lab4.asm`
- **Действие:** В файле `lab4.asm` я заменил строку `"Hello, world!"` на `"Ali Mukhtar Khadir"`.
- **Команда:** `nasm -f elf lab4.asm -o lab4.o` (Трансляция в объектный файл)
- **Команда:** `i686-linux-gnu-ld -m elf_i386 lab4.o -o lab4` (Компоновка через кросс-компоновщик)
- **Команда:** `qemu-i386 ./lab4` (Запуск программы через эмулятор)
- **Результат:** Программа выводит мое имя, что подтверждает корректность работы с данными.

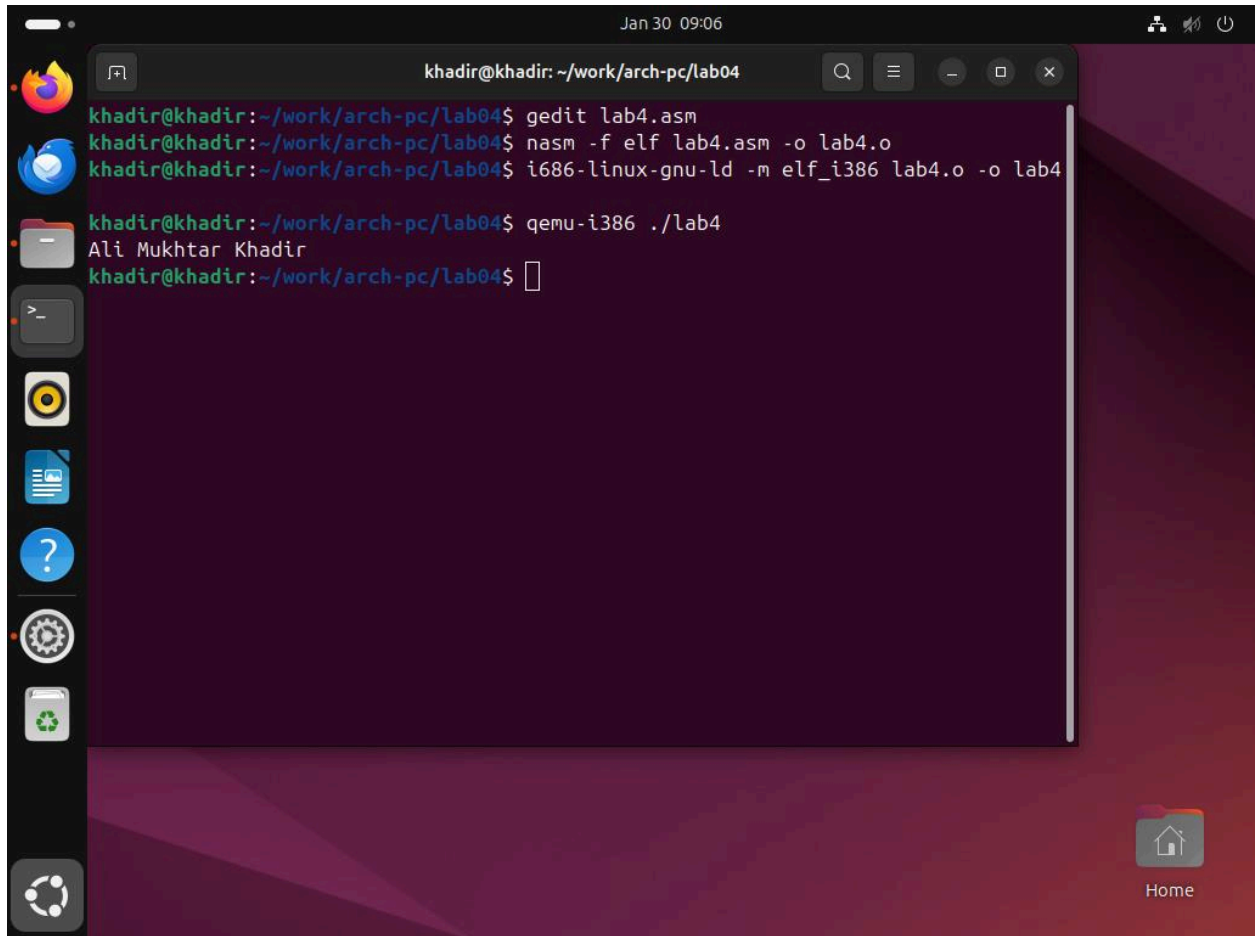


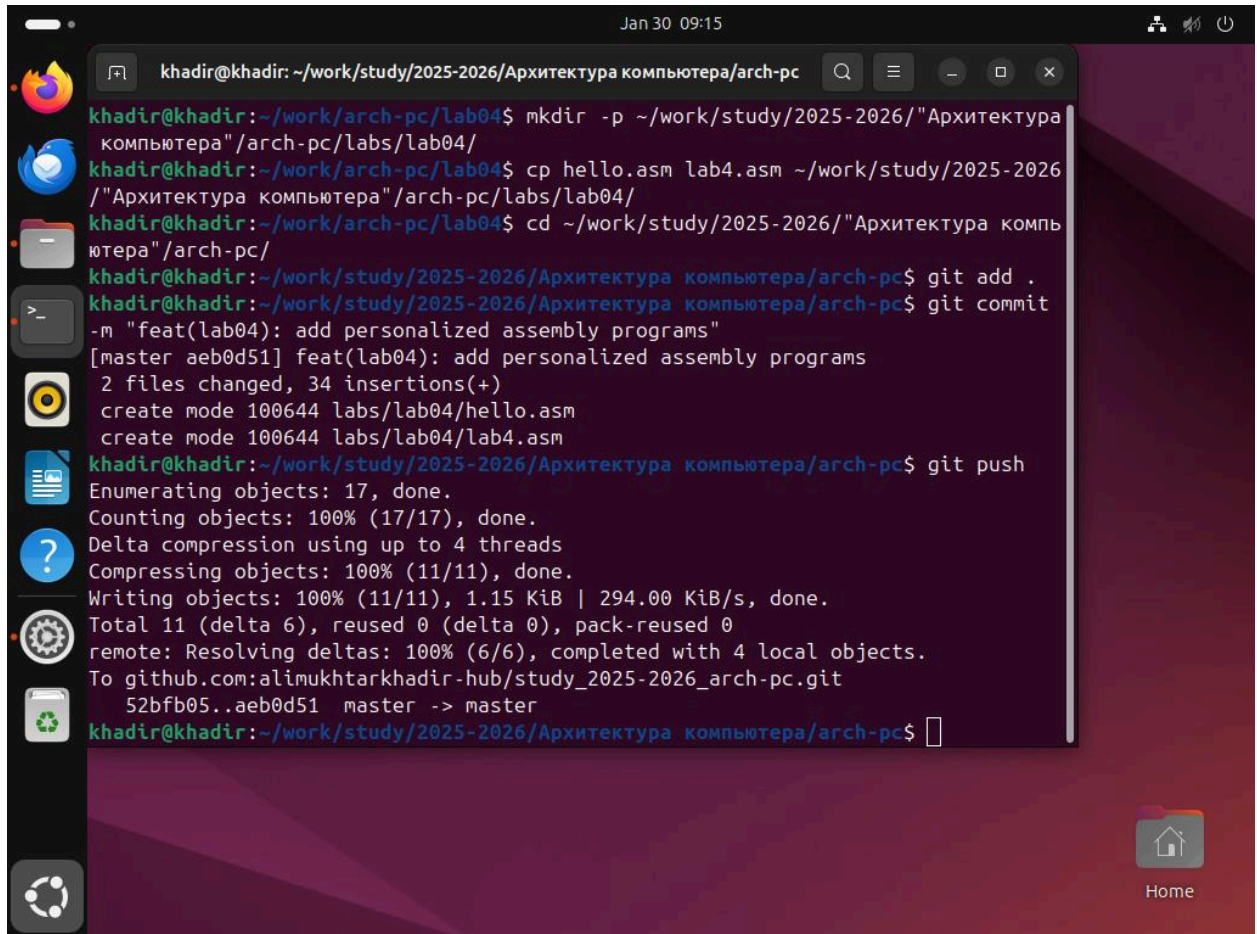
Рис. 4.4. Запуск персонализированной программы.

2.5. Синхронизация с репозиторием

Я переместил файлы лабораторной работы в локальный репозиторий и выполнил синхронизацию с GitHub.

- **Команда:** `mkdir -p ~/work/study/2025-2026/"Архитектура компьютера"/arch-pc/labs/lab04/`
- **Команда:** `cp hello.asm lab4.asm ~/work/study/2025-2026/"Архитектура компьютера"/arch-pc/labs/lab04/`
- **Команда:** `cd ~/work/study/2025-2026/"Архитектура компьютера"/arch-pc/`
- **Команда:** `git add .`

- **Команда:** `git commit -m "feat(lab04): add personalized assembly programs"`
- **Команда:** `git push`
- **Результат:** Файлы успешно загружены в удаленный репозиторий.



```
khadir@khadir: ~/work/study/2025-2026/Архитектура компьютера/arch-pc
khadir@khadir:~/work/arch-pc/lab04$ mkdir -p ~/work/study/2025-2026/"Архитектура
компьютера"/arch-pc/labs/lab04/
khadir@khadir:~/work/arch-pc/lab04$ cp hello.asm lab4.asm ~/work/study/2025-2026
/"Архитектура компьютера"/arch-pc/labs/lab04/
khadir@khadir:~/work/arch-pc/lab04$ cd ~/work/study/2025-2026/"Архитектура компь
ютера"/arch-pc/
khadir@khadir:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git add .
khadir@khadir:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git commit
-m "feat(lab04): add personalized assembly programs"
[master aeb0d51] feat(lab04): add personalized assembly programs
2 files changed, 34 insertions(+)
create mode 100644 labs/lab04/hello.asm
create mode 100644 labs/lab04/lab4.asm
khadir@khadir:~/work/study/2025-2026/Архитектура компьютера/arch-pc$ git push
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 4 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 1.15 KiB | 294.00 KiB/s, done.
Total 11 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), completed with 4 local objects.
To github.com:alimukhtarkhadir-hub/study_2025-2026_arch-pc.git
52bfb05..aeb0d51 master -> master
khadir@khadir:~/work/study/2025-2026/Архитектура компьютера/arch-pc$
```

Рис. 4.5. Загрузка файлов на GitHub.

3. Ответы на контрольные вопросы

1. **Отличие от языков высокого уровня?** Ассемблер — это язык низкого уровня, который работает напрямую с регистрами и памятью процессора.

2. **Инструкция vs Директива?** Инструкция (например, `mov`) преобразуется в машинный код для процессора; директива (например, `SECTION`) управляет процессом самой сборки.
3. **Правила оформления?** Одна команда на строку, чувствительность к регистру, обязательное разделение на секции данных и кода.
4. **Этапы получения файла?** Написание текста (`.asm`) -> Трансляция (`.o`) -> Компоновка (исполняемый файл) -> Запуск.
5. **Назначение трансляции?** Преобразование мнемоник (команд) ассемблера в машинный объектный код.
6. **Назначение компоновки?** Объединение объектных файлов в один готовый к работе исполняемый файл.
7. **Создаваемые файлы?** Объектный файл (`.o`) и файл листинга (`.lst`). По умолчанию создается только `.o`.
8. **Форматы файлов?** `nasm` использует формат ELF (32 или 64 бит), а `ld` использует ключи архитектуры, такие как `elf_i386`.

4. Вывод В ходе выполнения лабораторной работы №4 я ознакомился с процессом написания, трансляции и компоновки программ на языке ассемблера NASM. Мною была успешно решена проблема несовместимости архитектур процессора (ARM64 и x86) путем использования инструментов кросс-компиляции `i686-linux-gnu-ld` и эмулятора `qemu-i386`. Полученные навыки позволяют работать с низкоуровневым программным обеспечением и системами контроля версий.